



Universidad Tecnológica de Bolívar

UNIVERSIDAD TECNOLÓGICA DE BOLÍVAR

Proyecto de aula: Biblioteca Digital

Estudiantes:

German De Armas Castaño - T00068765

Angel Vega Rodriguez - T00068186

Profesor:

Rafael Enrique Monterroza Barrios

April 27, 2026

Contents

1	abstract	2
2	Introducción	2
3	Objetivos del Proyecto	2
3.1	Objetivo General	2
3.2	Objetivos Específicos	3
4	Métricas de Éxito	3
5	Entregables del Proyecto	3
6	Modelo Conceptual de Datos	4
7	Metodología de Desarrollo y Estructuras de Datos Implementadas	5
7.1	Árbol Binario de Búsqueda (BST) para el Catálogo de Libros	5
7.2	Tabla Hash para el Control de Usuarios	6
7.3	Listas Enlazadas para el Historial de Préstamos	9
8	Arquitectura del Sistema	10
9	Interacción entre Frontend y Backend	11
10	Implementación del Sistema	11
11	Conclusión	12

1 abstract

El presente documento constituye el informe técnico del sistema "Biblioteca Digital". Este proyecto tuvo como propósito principal diseñar y construir una plataforma cliente-servidor para la gestión eficiente de catálogos de libros, usuarios y préstamos en entornos bibliotecarios. El enfoque central radicó en la aplicación rigurosa de estructuras de datos algorítmicas computacionales con el fin de garantizar tiempos de respuesta óptimos durante las búsquedas y el almacenamiento de la información, objetivos que fueron logrados con éxito en la implementación final.

2 Introducción

El contexto actual de las bibliotecas, tanto físicas como digitales, exige una infraestructura tecnológica capaz de soportar operaciones concurrentes y manejar grandes volúmenes de información. La gestión paralela de miles de títulos de libros, registros de autores, historial de usuarios y flujos constantes de préstamos y devoluciones supone un reto significativo en términos de velocidad de procesamiento y organización en la memoria.

Tradicionalmente, la importancia de una gestión eficiente radica en la capacidad del sistema para responder a las consultas de los usuarios sin demoras perceptibles. La experiencia del usuario depende críticamente de componentes de software que sean capaces de acceder a colecciones de datos masivos en cuestión de milisegundos. Cuando un usuario de la biblioteca digital busca un libro por código o título, o cuando el administrador intenta validar la identidad y el estado de un prestatario, el sistema debe efectuar la búsqueda evitando recorrer toda la base de datos de manera secuencial, lo cual es altamente ineficiente y no escalable.

El problema fundamental de la búsqueda y organización de la información se intensifica a medida que el inventario de la biblioteca crece. Las búsquedas lineales u operaciones no optimizadas generan cuellos de botella que terminan degradando el rendimiento completo de la aplicación. Por esta razón, el diseño de este proyecto no recayó exclusivamente en un modelo relacional estándar, sino que implementó una simulación de almacenamiento en memoria a través de algoritmos óptimos, aplicando la teoría de estructuras de datos directamente en el núcleo de la aplicación.

El propósito central del sistema desarrollado, denominado "Biblioteca Digital", no solo fue construir una aplicación funcional como producto final, sino demostrar la aplicación práctica y efectiva de la algoritmia concurrente en un entorno real.

3 Objetivos del Proyecto

3.1 Objetivo General

Diseñar, planificar e implementar un sistema eficiente denominado "Biblioteca Digital", que integró estructuras de datos algorítmicas avanzadas dentro de una arquitectura cliente-servidor moderna, garantizando un ecosistema escalable y de alto rendimiento para la gestión integral de catálogos bibliográficos, usuarios y registro de préstamos.

3.2 Objetivos Específicos

- **Desarrollo fundamental del núcleo logístico:** Se diseñó una biblioteca digital orientada a objetos que abarca todos los modelos de negocio necesarios, como Libro, Autor, Editorial, Nacionalidad, Usuario y Préstamo.
- **Implementación teórica y práctica de estructuras de datos:** Se analizó e implementó el uso de distintas estructuras de datos para optimizar la gestión de la información. En particular, se utilizó un Árbol Binario de Búsqueda (BST) para el catálogo de libros, una Tabla Hash para el registro de usuarios y Listas Enlazadas para el manejo del historial de préstamos, reduciendo exitosamente la complejidad computacional.
- **Creación de la arquitectura cliente-servidor:** Se estableció un sistema desacoplado, desarrollando un servidor backend robusto basado en el ecosistema Python capaz de atender peticiones asíncronas mediante una API REST.
- **Desarrollo de interfaz de usuario web:** Se construyó un frontend moderno en Vite y React/JS que proporciona a los bibliotecarios un portal interactivo, dinámico y estéticamente profesional.
- **Optimización estricta de las búsquedas:** Se comprobó, a través de análisis de rendimiento empírico, que las operaciones de rescate de información y búsqueda alcanzaron cuotas de tiempo asintótico de $O(\log n)$ y $O(1)$ respectivamente, superando ampliamente a las implementaciones matriciales comunes.

4 Métricas de Éxito

Para garantizar que el desarrollo cumpliera con las especificaciones técnicas trazadas, se evaluó el sistema final contra una serie de métricas críticas que determinaron su viabilidad de despliegue en un entorno real:

- **Eficiencia de acceso a usuarios:** La métrica estipuló que la inserción, modificación o validación de un identificador numérico o cadena de usuario se completara con una complejidad de tiempo cercana a $O(1)$. Las pruebas confirmaron este rendimiento gracias a la Tabla Hash implementada.
- **Escalabilidad del sistema:** La aplicación demostró ser capaz de manejar operaciones concurrentes, como la incorporación de nuevos préstamos, sin generar conflictos asociados al redimensionamiento de estructuras dependientes ni provocar un uso ineficiente de la memoria RAM del servidor.

5 Entregables del Proyecto

El ciclo de desarrollo y diseño derivó en la entrega de los siguientes artefactos tecnológicos y académicos que validan la consecución completa del proyecto:

- **Código fuente del sistema:** Estructura de repositorios y carpetas contenidas que exhiben lógica legible, escalable y mantenible. Se entregó el paquete particionado y con sus configuraciones de empaquetado ('requirements.txt' y 'package.json').

- **Documentación técnica e Informe final:** El presente documento ampliado, el cual sustenta las especificaciones, modelos UML, arquitectura implementada y decisiones operativas fundamentales.
- **Manual Técnico y Manual de Usuario Funcional:** Documentación específica que guía el despliegue del entorno y el uso de la plataforma interactiva.

6 Modelo Conceptual de Datos

Antes de proceder con la selección e implementación de estructuras de datos algorítmicas, resultó necesario definir la forma en la que la información del sistema se organizaría desde una perspectiva conceptual. Para tal fin, se planteó un modelo de datos que describe las entidades principales del dominio bibliotecario y las relaciones existentes entre ellas.

La Figura 1 presenta el diagrama conceptual que estructuró la base de datos del sistema desarrollado. En dicho esquema se identifican las entidades fundamentales que participan en la gestión de la biblioteca digital, así como sus atributos y las relaciones que las vinculan.

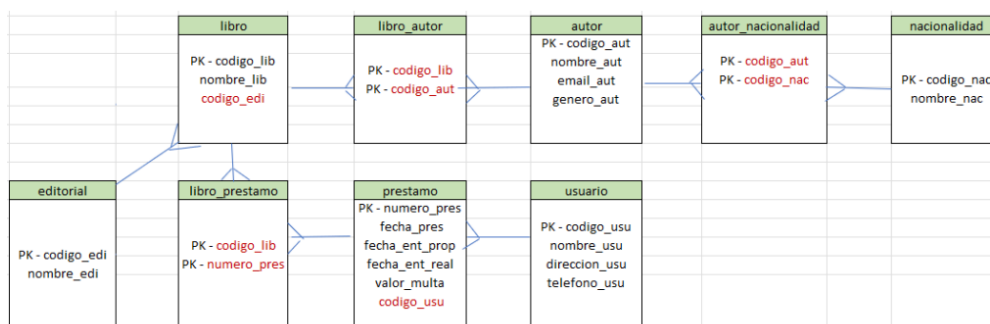


Figure 1: Modelo conceptual de datos del sistema Biblioteca Digital

El modelo conceptual define varias entidades principales. En primer lugar, la entidad *Libro* representa cada uno de los títulos almacenados dentro del catálogo. Cada libro se encuentra asociado a una *Editorial*, lo cual permite registrar la casa editorial responsable de su publicación.

Por otra parte, la entidad *Autor* almacena la información de los escritores. Dado que un libro puede tener múltiples autores y un autor puede haber participado en la creación de varios libros, esta relación se modeló mediante una tabla intermedia denominada *Libro_Autor*, la cual permitió representar adecuadamente esta relación de muchos a muchos.

Asimismo, el sistema contempla la entidad *Usuario*, la cual representa a las personas que interactúan con la biblioteca y solicitan préstamos de material bibliográfico. Cada usuario puede generar múltiples registros de préstamo a lo largo del tiempo.

La entidad *Prestamo* registra las transacciones realizadas dentro del sistema, incluyendo la fecha en la que se realiza el préstamo, la fecha estimada de devolución, la fecha real de entrega y cualquier posible multa asociada a retrasos. Debido a que un préstamo puede incluir varios libros simultáneamente, se introdujo una tabla intermedia *Libro_Prestamo*, representando la relación entre los préstamos y los libros involucrados.

Finalmente, la entidad *Nacionalidad* permite clasificar a los autores según su país de origen, lo que facilita la organización y consulta de información cultural o geográfica dentro del catálogo.

Este modelo conceptual constituyó la base estructural sobre la cual se diseñó la lógica del sistema y la implementación de las estructuras de datos propuestas.

7 Metodología de Desarrollo y Estructuras de Datos Implementadas

El corazón del proyecto "Biblioteca Digital" se fundamentó en no delegar la algoritmia a un motor relacional de base de datos genérico, sino en modelar activamente el almacenamiento de datos en el nivel aplicativo, logrando una eficiencia técnica superior. A continuación, se detallan las estructuras implementadas:

7.1 Árbol Binario de Búsqueda (BST) para el Catálogo de Libros

¿Qué es y cómo funciona en el sistema?: Un BST (Binary Search Tree) es una estructura jerárquica dirigida donde cada nodo tiene a lo sumo dos hijos. En nuestro sistema, cada nodo almacena un objeto *Libro*. Al insertar un nuevo libro o buscar uno existente, el sistema compara su identificador único (ID o título) con la raíz y desciende por la rama izquierda (si es menor) o derecha (si es mayor) hasta encontrar su posición o el dato buscado.

¿Por qué se eligió y complejidad computacional?: El catálogo de libros es extenso y las consultas de disponibilidad superan ampliamente a las inserciones de nuevos volúmenes. El BST proveyó una complejidad asintótica promedio de $O(\log n)$ para búsqueda, inserción y eliminación. Esto resolvió la problemática de estancamiento en las búsquedas lineales tradicionales al ubicar libros desde el frontend.

Implementación en código:

```
class NodoArbol:
    def __init__(self, clave, valor):
        self.clave = clave      # El codigo_lib será la clave
        self.valor = valor     # El objeto Libro será el valor
        self.izquierdo = None
        self.derecho = None

class ArbolBinarioBusqueda:
    def __init__(self):
        self.raiz = None

    def insertar(self, clave, valor):
        if self.raiz is None:
            self.raiz = NodoArbol(clave, valor)
        else:
            self._insertar_recursivo(self.raiz, clave, valor)

    def _insertar_recursivo(self, nodo_actual, clave, valor):
```

```

    if clave < nodo_actual.clave:
        if nodo_actual.izquierdo is None:
            nodo_actual.izquierdo = NodoArbol(clave, valor)
        else:
            self._insertar_recursivo(nodo_actual.izquierdo, clave, valor)
    elif clave > nodo_actual.clave:
        if nodo_actual.derecho is None:
            nodo_actual.derecho = NodoArbol(clave, valor)
        else:
            self._insertar_recursivo(nodo_actual.derecho, clave, valor)
    else:
        # Si la clave ya existe, se actualiza el valor (o se podría manejar como e
        nodo_actual.valor = valor

def buscar(self, clave):
    return self._buscar_recursivo(self.raiz, clave)

def _buscar_recursivo(self, nodo_actual, clave):
    if nodo_actual is None:
        return None
    if nodo_actual.clave == clave:
        return nodo_actual.valor
    elif clave < nodo_actual.clave:
        return self._buscar_recursivo(nodo_actual.izquierdo, clave)
    else:
        return self._buscar_recursivo(nodo_actual.derecho, clave)

def recorrido_inorden(self):
    resultado = []
    self._recorrido_inorden_recursivo(self.raiz, resultado)
    return resultado

def _recorrido_inorden_recursivo(self, nodo_actual, resultado):
    if nodo_actual is not None:
        self._recorrido_inorden_recursivo(nodo_actual.izquierdo, resultado)
        resultado.append(nodo_actual.valor)
        self._recorrido_inorden_recursivo(nodo_actual.derecho, resultado)

```

7.2 Tabla Hash para el Control de Usuarios

¿Qué es y cómo funciona en el sistema?: Es una estructura de datos que asocia claves a valores mediante una función matemática (función hash). En la biblioteca, el documento de identidad alfanumérico del usuario actúa como clave. La función hash computa un índice en un arreglo principal (buckets) donde se almacena el objeto **Usuario**. Las colisiones se manejaron mediante encadenamiento, adosando Listas Enlazadas en las ranuras colisionadas.

¿Por qué se eligió y complejidad computacional?: Durante procesos críticos y

de alto tráfico (como préstamos y devoluciones rápidas), es inaceptable tener que recorrer listas secuenciales para validar a un usuario. La Tabla Hash permitió recuperar el perfil del usuario en un tiempo constante $O(1)$, maximizando la respuesta del servidor sin importar el volumen demográfico de usuarios registrados.

Implementación en código:

```
class NodoHash:
    def __init__(self, clave, valor):
        self.clave = clave      # El codigo_usu
        self.valor = valor     # Objeto Usuario
        self.siguiente = None  # Para encadenamiento en caso de colisión

class TablaHash:
    def __init__(self, tamano=100):
        self.tamano = tamano
        self.tabla = [None] * tamano
        self.num_elementos = 0

    def _hash(self, clave):
        # Una función hash simple basada en el valor de la clave (string o int)
        if isinstance(clave, str):
            h = 0
            for caracter in clave:
                h = (h * 31 + ord(caracter)) % self.tamano
            return h
        elif isinstance(clave, int):
            return clave % self.tamano
        else:
            return hash(clave) % self.tamano

    def insertar(self, clave, valor):
        indice = self._hash(clave)

        # Revisar si la clave ya existe para actualizar el valor
        nodo_actual = self.tabla[indice]
        while nodo_actual is not None:
            if nodo_actual.clave == clave:
                nodo_actual.valor = valor
                return
            nodo_actual = nodo_actual.siguiente

        # Insertar al inicio de la lista enlazada (encadenamiento)
        nuevo_nodo = NodoHash(clave, valor)
        nuevo_nodo.siguiente = self.tabla[indice]
        self.tabla[indice] = nuevo_nodo
        self.num_elementos += 1

    # Redimensionamiento dinámico (factor de carga > 0.7)
```



```

        if self.num_elementos / self.tamano > 0.7:
            self._redimensionar()

    def buscar(self, clave):
        indice = self._hash(clave)
        nodo_actual = self.tabla[indice]
        while nodo_actual is not None:
            if nodo_actual.clave == clave:
                return nodo_actual.valor
            nodo_actual = nodo_actual.siguiente
        return None

    def eliminar(self, clave):
        indice = self._hash(clave)
        nodo_actual = self.tabla[indice]
        nodo_anterior = None

        while nodo_actual is not None:
            if nodo_actual.clave == clave:
                if nodo_anterior is None:
                    self.tabla[indice] = nodo_actual.siguiente
                else:
                    nodo_anterior.siguiente = nodo_actual.siguiente
                self.num_elementos -= 1
                return True
            nodo_anterior = nodo_actual
            nodo_actual = nodo_actual.siguiente
        return False

    def _redimensionar(self):
        nuevo_tamano = self.tamano * 2
        nueva_tabla = [None] * nuevo_tamano

        # Mover todos los elementos
        for i in range(self.tamano):
            nodo_actual = self.tabla[i]
            while nodo_actual is not None:
                indice = self._hash(nodo_actual.clave)
                nuevo_nodo = NodoHash(nodo_actual.clave, nodo_actual.valor)
                nuevo_nodo.siguiente = nueva_tabla[indice]
                nueva_tabla[indice] = nuevo_nodo
                nodo_actual = nodo_actual.siguiente

        self.tamano = nuevo_tamano
        self.tabla = nueva_tabla

    def obtener_todos(self):

```

```

elementos = []
for i in range(self.tamano):
    nodo_actual = self.tabla[i]
    while nodo_actual is not None:
        elementos.append(nodo_actual.valor)
        nodo_actual = nodo_actual.siguiente
return elementos

```

7.3 Listas Enlazadas para el Historial de Préstamos

¿Qué es y cómo funciona en el sistema?: Una Lista Enlazada es un modelo lineal que almacena registros discretos enlazados lógicamente por apuntadores, sin necesidad de contigüidad en memoria. En el sistema, cada vez que se efectúa un préstamo, se crea un bloque (nodo) que contiene los detalles de la transacción (fecha, usuario, libros arrendados) y un puntero al siguiente préstamo, originando un cordón transaccional histórico inmutable.

¿Por qué se eligió y complejidad computacional?: El historial de transacciones requiere inserciones constantes y rápidas al final de la cola temporal, priorizando la inserción en el fondo de la línea en tiempo $O(1)$. La lista enlazada evitó el costoso redimensionamiento y copia de arreglos estáticos cuando la memoria se rebosa, dotando al sistema de un registro auditable perfecto para el control de caducidades y multas.

Implementación en código:

```

class NodoLista:
    def __init__(self, valor):
        self.valor = valor          # Objeto Prestamo
        self.siguiente = None

class ListaEnlazada:
    def __init__(self):
        self.cabeza = None
        self cola = None           # Mantenemos referencia a la cola para inserción O(1)
        self.tamano = 0

    def agregar_al_final(self, valor):
        nuevo_nodo = NodoLista(valor)
        if self.cabeza is None:
            self.cabeza = nuevo_nodo
            self.col = nuevo_nodo
        else:
            self.col.siguiente = nuevo_nodo
            self.col = nuevo_nodo
        self.tamano += 1

    def agregar_al_inicio(self, valor):
        nuevo_nodo = NodoLista(valor)
        if self.cabeza is None:
            self.cabeza = nuevo_nodo

```

```

        self.cola = nuevo_nodo
    else:
        nuevo_nodo.siguiente = self.cabeza
        self.cabeza = nuevo_nodo
    self.tamano += 1

def buscar(self, condicion):
    # La condición es una función lambda que devuelve True si el elemento coincide
    resultados = []
    nodo_actual = self.cabeza
    while nodo_actual is not None:
        if condicion(nodo_actual.valor):
            resultados.append(nodo_actual.valor)
        nodo_actual = nodo_actual.siguiente
    return resultados

def obtener_todos(self):
    elementos = []
    nodo_actual = self.cabeza
    while nodo_actual is not None:
        elementos.append(nodo_actual.valor)
        nodo_actual = nodo_actual.siguiente
    return elementos

def esta_vacia(self):
    return self.cabeza is None

def __len__(self):
    return self.tamano

```

8 Arquitectura del Sistema

El desarrollo del proyecto se fundamentó en una arquitectura cliente-servidor fuertemente desacoplada, lo que permitió separar la lógica de negocio y el almacenamiento algorítmico en memoria de la presentación visual al usuario.

Arquitectura Cliente-Servidor y API REST: El sistema fue dividido en dos componentes lógicos autónomos. El Backend actuó como el núcleo lógico (servidor), gestionando las estructuras de datos y respondiendo a peticiones mediante una API RESTful estandarizada. El Frontend (cliente) se encargó de la renderización y la interacción con el usuario final.

Tecnologías Utilizadas:

- **Backend:** Implementado en el ecosistema Python, utilizando frameworks para la exposición de endpoints HTTP. Toda la persistencia en memoria y algoritmia concurrente fue construida y controlada directamente a través de código Python.
- **Frontend:** Desarrollado con JavaScript moderno empleando el empaquetador Vite

y librerías de renderizado reactivo, garantizando una interfaz fluida (SPA - Single Page Application).

Flujo de Datos: Cuando un usuario realiza una acción en la interfaz, el frontend despacha una solicitud HTTP al backend. El servidor procesa la información consultando o modificando el BST, la Tabla Hash o las Listas Enlazadas. Finalmente, el servidor empaqueta los resultados en formato JSON normalizado y los devuelve al cliente, quien actualiza la vista reactivamente.

9 Interacción entre Frontend y Backend

La comunicación y el acoplamiento entre las dos capas del proyecto se realizó de manera asíncrona mediante el consumo directo de la API REST por parte del frontend.

Endpoints Implementados: Se definieron rutas semánticas que respetan los estándares HTTP:

- GET /libros: Retorna el catálogo completo recorriendo el BST.
- GET /usuarios/{id}: Consulta el perfil de un usuario específico en tiempo $O(1)$ gracias a la Tabla Hash.
- POST /prestamos: Registra una nueva transacción de arrendamiento insertando un nodo en la Lista Enlazada.

Flujo Completo de un Préstamo: 1. **Usuario:** El bibliotecario llena el formulario de préstamo en la pantalla y presiona el botón de registro. 2. **Frontend:** El sistema captura los datos del estado de la vista y emite una solicitud POST. 3. **Backend:** Python recibe el payload JSON, verifica la identidad y el estado activo del usuario en la Tabla Hash (en coste $O(1)$) y confirma la disponibilidad física de cada libro re-ruteando la búsqueda en el árbol BST (en coste $O(\log n)$). 4. **Estructuras de Datos:** Si las validaciones son exitosas, se instancia el modelo **Prestamo** con su marca temporal ISO y se añade un nuevo eslabón a la cabeza de la Lista Enlazada. Se descuenta entonces el stock del libro. 5. **Respuesta:** El servidor retorna un estado HTTP 201 **Created**. El frontend intercepta la respuesta afirmativa y redibuja su tabla visual de préstamos de manera instantánea.

Ejemplo de Petición HTTP (POST):

```
POST /prestamos HTTP/1.1
Content-Type: application/json

{
  "usuario_id": "T00068765",
  "libros": [105, 204],
  "fecha_prestamo": "2026-04-25T10:00:00Z"
}
```

10 Implementación del Sistema

El engranaje del sistema funciona a un ritmo secuencial y altamente optimizado. Al inicializarse el servidor backend de Python, se instancian los repositorios lógicos alojando

en memoria la raíz vacía del árbol BST, el vector inicial de n -casillas de la Tabla Hash, y los apuntadores nulos de la Lista Enlazada.

Para permitir la compatibilidad entre el lenguaje de servidor con el ecosistema de red, los modelos nativos de las entidades (como el registro **Libro** que alberga variables internas y un arreglo adjunto referencial de **Autor**) son procesados rigurosamente en diccionarios serializables de Python mediante funciones intrínsecas, retornando siempre estructuras normalizadas.

El ecosistema Frontend se articula dentro del servidor local montado mediante Vite, consumiendo e interactuando dinámicamente con estas estructuras.

Si un bibliotecario decide arrendar o generar un préstamo transaccional, el controlador en Python valida que el código primario del usuario colisione satisfactoriamente en la Tabla Hash con un tiempo nominal plano de $O(1)$. Si el usuario se halla en estado activo, el sistema bifurca una búsqueda logarítmica por el BST hasta verificar la presencia real y física de cada uno de los libros listados. Tras esta doble aprobación veloz, se graba una marca temporal y se añade de manera irrecusable un nuevo eslabón de memoria a la Lista Enlazada. Posteriormente, el backend retorna un estado afirmativo HTTP al hilo del Frontend, que redibuja su estática de préstamos culminando el recorrido en una faceta asombrosamente expedita y fluida.

11 Conclusión

En síntesis, este documento ha presentado los resultados finales y la envergadura técnica del proyecto "Biblioteca Digital". Con la ejecución exitosa de los lineamientos de arquitectura propuestos, se dio a luz no simplemente a un gestor informático tradicional o a un software base relacional genérico, sino a una obra de alto nivel de complejidad diseñada y construida para maximizar la eficiencia funcional.

La evaluación demostró que el cumplimiento de los objetivos fue categórico: se lograron tiempos de búsqueda rápidos asintóticos de $O(\log n)$ en el catálogo general, y procesos constantes de $O(1)$ en el área de perfiles e inserción del historial. La integración precisa de un BST, una Tabla Hash y Listas Enlazadas directamente en la memoria del aplicativo garantizó flujos operativos estables y optimización radical del rendimiento del servidor, sentando un precedente sobre la aplicación práctica de la algoritmia en arquitecturas web cliente-servidor.